

Z3 tutorial

Chenxi Ji

Type of Variables

- Int, Bool, Real

```
x = Int('x')
```

```
x = Bool('x')
```

```
x = Real('x')
```

Define Math Expression and Simplify It

```
x = Int('x')
y = Int('y')
print (simplify(x + y + 2*x + 3))
print (simplify(x < y + x + 2))
print (simplify(And(x + 1 >= 3, x**2 + x**2 + y**2 + 2 >= 5)))
```

Define Constraints and Solve a Solution

```
x = Int('x')  
y = Int('y')  
solve(x>2, y < 10, x+2*y==7)  
solve(x>4, x<-1)
```

Solve Boolean Constraints

```
p = Bool('p')
```

```
q = Bool('q')
```

```
r = Bool('r')
```

```
solve(Implies(p, q), r == Not(q), Or(Not(p), r))
```

Dijkstra – Bound function

- Math Expression: For all i in $[0, N-1]$, $0 \leq x[i] < K$
- Steps:
 - Construct a list to store all constraints
 - Provide constraints for each $x[i]$: $\text{And}(x[i] \geq 0, x[i] < K)$
 - Combine all constraints together (And relation)

Dijkstra – has_token function

- Math Expression:
 - If $i == 0$ and $x[i] == x[N-1]$, then return True
 - If $i != 0$ and $x[i] != x[i-1]$, then return True
- Steps:
 - Consider case 1: $\text{And}(i == 0, x[i] == x[N-1])$
 - Consider case 2: $\text{And}(\text{Not}(i == 0), \text{Not}(x[i] == x[i-1]))$
 - Combine two cases (Or relation)

Dijkstra – legal_config function

- Math Expression:
 - Sum of $\text{has_token}(x, i)$ for all i in $[0, N-1]$, denote as Args . If $\text{Args}==1$, then return True; else return False
- Steps:
 - Generate a list of output of has_token function:
 - $\text{args} = [\text{has_token}(x, i) \text{ for } i \text{ in range}(0, N)]$
 - Sum of $\text{args} \leq 1$: $\text{AtMost}(\text{args} + [1])$
 - Sum of $\text{args} \geq 1$: $\text{AtLeast}(\text{args} + [1])$
 - Combine them together(And relation)

Dijkstra – transition_relation function

- Math Expression:
 - If node 0 has_token, then $\text{new_x}[0] == (\text{old_x}[0] + 1) \% K$
 - Else for node 0, $\text{new_x}[0] == \text{old_x}[0]$
 - If node $i \neq 0$ has token, then $\text{new_x}[i] == \text{old_x}[i-1]$
 - Else for node $i \neq 0$, then $\text{new_x}[i] == \text{old_x}[i]$
- Steps :
 - $\text{Case0_1} = \text{And}(\text{has_token}(\text{old_x}, 0), \text{new_x}[0] == (\text{old_x}[0] + 1) \% K)$
 - $\text{Case0_2} = \text{And}(\text{Not}(\text{has_token}(\text{old_x}, 0)), \text{new_x}[0] == \text{old_x}[0])$
 - $\text{Node_0} = \text{Or}(\text{Case0_1}, \text{Case0_2})$
 - $\text{Casei_1} = \text{And}(\text{has_token}(\text{old_x}, i), \text{new_x}[i] == \text{old_x}[i-1])$
 - $\text{Casei_2} = \text{And}(\text{Not}(\text{has_token}(\text{old_x}, i)), \text{new_x}[i] == \text{old_x}[i])$
 - $\text{Node_i} = \text{Or}(\text{Casei_1}, \text{Casei_2})$
 - Combine constraints on all nodes together(And relation)